GENZONE, AI Manthan

TurboZone: 100 questions a judge may ask, with our answers

Answers use the numbers from the final training run (nine N-CMAPSS subsets, 39 held-out test engines), the 19-check physics check and the 69 backend tests. Say them in your own words; the facts are what matter.

How to use this

- **Prepare the ten marked HARD questions first.** Each targets a real limitation. Judges reward a team that names its limits plainly before being pushed, so answer them directly and without defensiveness.
- Lead with the one-sentence answer, then give one number. Stop there unless asked for more.
- Highlighted answers (questions 92, 98 and 99) need the team's own details before the presentation.

Problem and impact	1–10	Results and honesty	51–60
Data	11–20	Explainability and trust	61–70
Physics layer	21–30	Product and dashboard	71–80
Combustor and nozzle physics	31–40	Engineering and deployment	81–90
Machine learning	41–50	Team, novelty and future	91–100

Problem and impact Questions 1–10

1 What problem are you solving, and who exactly is the user?

We predict how many flights each turbofan engine has left before it reaches its failure level, and which module is wearing. The user is the maintenance engineer or planner at an airline or MRO. TurboZone turns the sensor data from every flight into a ranked work queue: which engine to look at first, why, and by when the shop visit should happen.

2 Why does predicting remaining useful life matter to an airline in money terms?

An unscheduled engine removal is one of the most expensive events in airline operations: a shop visit, a spare or leased engine, and an aircraft on the ground with cancelled flights. Predicting RUL lets planners move removals into planned downtime and avoid both premature removals, which waste remaining life, and in-service failures. We have not modelled costs ourselves, so we talk about the mechanism (unscheduled becomes scheduled) rather than a dollar figure.

3 **How does your system change what a maintenance engineer does on a Monday morning?**

They open the fleet queue, which is already ranked by urgency. The top engine shows its planning RUL, the most degraded module, suggested next steps and open alerts. They book a borescope or a shop visit, acknowledge the alert and record what they found. That finding is stored next to the prediction that was shown, so the model can later be checked against reality.

4 **What happens today without a tool like this?**

Engines are monitored by trending a few parameters (EGT margin, fuel flow, vibration) against fixed limits, plus inspections scheduled by hours or cycles. Engineers read trend plots by eye. It is hard to compare flights at different altitudes and throttle settings, limits often trigger late, and there is no explicit uncertainty on how long is left.

5 **Why a digital twin rather than just a prediction model?**

Because we keep a physical state of each engine that updates every flight: a healthy-engine baseline for the same flight condition, a Kalman filter estimating each module's efficiency and flow capacity, and physics monitors for the combustor and nozzle. That gives component-level health, explanations in engineering terms (which station is hotter than a healthy engine) and physical consistency checks. The RUL model sits on top of that state.

6 **Who would pay for this: an airline, an MRO shop or an engine manufacturer?**

Most naturally airline engineering departments and MROs, who plan shop visits and carry the cost of surprises; lessors care about asset value too. Engine manufacturers already run their own monitoring, so for them this is a reference design rather than a product.

7 **What decision does the planning RUL number support, and why is it the low end of the range?**

It answers 'by when must this engine be in the shop?'. We use the 5th-percentile bound of the 90% range because the costs are asymmetric: running out of life is unsafe and expensive, scheduling a few flights early costs a little life. It also protects against the model's mid-life bias, where it says on average 3.2 flights more than reality.

8 **If your tool is wrong, what's the cost of a false alarm versus a missed failure?** HARD

A missed failure can mean an in-flight shutdown or an unscheduled removal: a safety and cost event. A false alarm usually costs an unnecessary borescope, a few hours of labour. So we deliberately lean towards catching problems: planning uses the low end of the range, urgency takes the worse of component health and RUL, and alerts ask for an inspection, not a removal. On held-out engines the 90% range contained the true RUL on 93.4% of flights, and the error near end of life is 4.2 flights.

9 **How is this different from the engine health monitoring that GE, Rolls-Royce and Pratt & Whitney already sell?**

We do not claim to be more accurate: they have real fleet data and proprietary models. Our difference is transparency. Every number can be traced to a residual, a filter state or a SHAP value; uncertainty is shown, not hidden; a physics self-check runs on the data; and an inspection-feedback loop is built in. It is an open prototype on simulated data, not a certified product.

10 **What would it take for a real airline to trust and adopt this?**

Validation on their own engines: back-testing against past removals and shop findings, refitting the healthy baseline on their fleet, and recalibrating the uncertainty ranges. Then a shadow-mode period where predictions are logged and compared with findings without driving decisions, and integration with their data feeds and maintenance system.

Data Questions 11–20

11 **What is N-CMAPSS, and how is it different from the original C-MAPSS?**

N-CMAPSS is NASA's 2021 run-to-failure dataset (Arias Chao et al.), generated with the CMAPSS turbofan model driven by real recorded flight profiles: complete flights at 1 Hz with realistic altitude, Mach and throttle, several failure modes, and ground truth for ten health parameters. The original C-MAPSS (2008) gives one snapshot per cycle under a few fixed operating conditions.

12 **Why did you choose N-CMAPSS over other datasets?**

It has realistic flight conditions, full run-to-failure histories, and ground truth for each module's degradation, which lets us validate our physics layer and not only the RUL. It is public and citable, and its subsets cover several failure modes.

13 **Your data is simulated. How do you know any of this works on real engines?** HARD

We don't know yet, and we say so. What transfers is the method: a healthy-engine baseline per engine type, a Kalman filter on gas-path physics, and monitors built on conservation laws are standard gas-path analysis practice. What must be redone on real data is the healthy baseline, the noise model, the uncertainty ranges and the health limits, followed by validation against real removal records. We designed for that: the inputs are only sensors a real engine has, never the simulator's virtual sensors.

14 **How many engines and flights did you train, validate and test on?**

99 engines and 7,473 flights across 9 subsets. Before anything was fitted we split by engine: 49 training engines (3,716 flights), 11 validation engines (819 flights) and NASA's 39 official test engines (2,938 flights).

15 **Why did you exclude DS08d?**

The Kaggle copy of that file is truncated inside the HDF5 file's internal index, so it cannot be read even after padding. We used the other nine files rather than work with a damaged one.

16 **What does one row of your data represent, and why aggregate per flight instead of using the 1 Hz data?**

One row is one completed flight: the mean, standard deviation, minimum, maximum and last value of each sensor, plus the flight conditions. RUL is defined per flight and maintenance decisions are per flight; aggregation removes 1 Hz noise and keeps millions of rows manageable. Raw 1 Hz uploads are aggregated the same way.

17 **What are the virtual sensors (T40, P30, P45), and why don't you use them as model inputs?**

They are quantities the simulator computes but a real engine does not measure: turbine inlet temperature, HPC exit total pressure and LPT inlet pressure. Using them would make the system impossible to deploy. We use them only for display and to validate our physics; for example our spool-balance estimate of T40 correlates 0.995 with the simulator's value.

18 **How do you handle different flight conditions (altitude, Mach, throttle)?**

With a healthy-engine baseline: for each of the 14 sensors, a degree-3 polynomial ridge model of altitude, Mach, throttle and inlet temperature, fitted on healthy flights of training engines. It predicts what a new engine would read on this flight; the residual (measured / healthy - 1) removes the flight-condition effect. Healthy residuals stay near zero in every altitude band.

19 **What does the healthy flag (hs) mean, and how did you use it?**

It is N-CMAPSS's label for the initial healthy period of each engine. We used it only to choose which flights fit the healthy baseline (training engines) and to evaluate the monitors in the physics check. It is never a model input.

20 **Did any information from test engines leak into training? How did you prevent it?** HARD

We split by engine before fitting anything. Test engines are NASA's official test set, used once at the end. The baseline, filter calibration and feature imputation are fitted on training engines; tuning, early stopping and the blend weight use validation engines. Training-engine residuals are cross-fitted, features only look backwards in time, and an audit blocks columns like RUL, unit and hs. One documented caveat: the filter's calibration is shared across training folds, so the held-out test metrics are the clean ones.

Physics layer Questions 21–30

21 **What is the healthy-engine baseline, and how is it fitted?**

A model of what each sensor would read on a new engine at a given flight condition. For each sensor it is a ridge regression on degree-3 polynomial terms of altitude, Mach, throttle and inlet temperature, fitted on healthy flights of training engines only.

22 **Why use residuals instead of raw sensor values?**

Raw sensors swing far more with altitude and throttle than with wear, so wear is buried. The residual shows only the part a healthy engine would not explain. Our ablation measures it: raw sensors give an RMSE of 10.2 flights, residuals bring it to 7.4.

23 **Your ablation shows residuals cut the error from 10.4 to 7.4 flights. Why such a big effect?**

Because flight condition dominates the raw sensor variance. A tree model on raw values spends its capacity relearning thermodynamics; with residuals the wear signal, such as T48 and T50 running hotter than a healthy engine, is directly visible.

24 **What does the Kalman filter estimate, and why an extended Kalman filter?**

It estimates the efficiency and flow-capacity modifiers of the fan, LPC, HPC, HPT and LPT (10 parameters, merged into 8 identifiable states) from each flight's residuals, with an uncertainty for each. It is 'extended' because the sensor response to those parameters depends on flight condition, so the model is linearised at each flight's operating point.

25 **Why does the filter merge some states, and what does 'not separable' mean physically?**

Some parameters move the 14 sensors in almost the same pattern (signature similarity of 0.95 or more, or little unique variance), so no amount of data can tell them apart. For example a fan efficiency loss and an LPT efficiency loss both show up mainly on the low-pressure spool. We merge such pairs into one state and report those modules as a group.

26 **Only LPC and HPC are identifiable. Doesn't that make the component-health claim weak?** HARD

It limits it, and we show that openly. LPC and HPC are isolated with a typical error of 4 to 7 points out of 100; fan, HPT and LPT have 13 to 17 points of error and are shown as groups. That is a limit of the sensor set, not a bug: separating them needs more turbine instrumentation. The dashboard labels them 'grouped' and asks for borescope, EGT margin and vibration data to pin down the module. The RUL prediction does not depend on isolation.

27 **Your filter's mean NIS is 26.5 against an expected 14. What does that tell you?**

The filter is overconfident: its prediction errors are larger than its own uncertainty says, which points to model mismatch or underestimated noise. So the plus-or-minus values it reports are too narrow; that is why component reliability in the dashboard comes from measured validation errors, not from the filter's own sigma. The fix would be noise inflation or an adaptive filter.

28 **How do you map filter estimates to a 0-1 health index, and what is the failure level?**

Damage is the larger of the module's efficiency and flow modifiers; health = 1 - damage / failure-level damage. The failure level is the 95th percentile of end-of-life damage in training engines (fan 0.060, LPC 0.041, HPC 0.048, HPT 0.019, LPT 0.030). Bands: Normal above 0.75, Warning above 0.55, Alarm above 0.30, Emergency below.

29 **What is a gated or rejected sensor reading, and why does it matter?**

If a sensor's normalised innovation exceeds 9 (three standard deviations), the filter rejects that reading for that flight so a spike cannot corrupt the health estimate. Repeated rejections raise a 'sensor health' alert. On validation engines 1.5% of readings were gated.

30 **How would the filter handle a faulty sensor rather than a faulty component?**

A sensor fault moves one sensor in a way the physics cannot explain, so its innovation is large and the reading is rejected; repeated rejections raise a sensor alert. A component fault moves several sensors consistently and is absorbed into the health states. It is a first line of defence, not a full sensor-fault isolation system.

Combustor and nozzle physics Questions 31–40

31 **N-CMAPSS never degrades the combustor. How are you 'tracking' it?**

With physics monitors: energy-balance and compressible-flow formulas evaluated each flight on measured sensors and compared with the same formula on the healthy reference. The deviation maps to health through a tolerance and a failure limit. In N-CMAPSS the combustor correctly stays at 1.00, which the physics check confirms (the simulator holds burner pressure loss at exactly 4.50%), and fault injection shows the monitors would detect a real change.

32 **Walk us through the combustion-efficiency formula.**

First, turbine inlet temperature from the HP-spool power balance. Second, core mass flow through the choked HPT guide vanes, proportional to P40 / sqrt(T40) and corrected for HPT flow capacity. Third, combustion efficiency = mass flow x enthalpy rise from T30 to T40 / (fuel flow x 43.1 MJ/kg), with temperature-dependent air properties. Dividing by the same formula on a healthy engine cancels the unknown constants. Injecting 3% extra fuel reads back as -2.91%, exactly as expected.

33 **How do you get turbine inlet temperature when T40 isn't measured?**

From energy conservation on the high-pressure spool: the HPT drives the HPC, so the enthalpy drop from T40 to T48 equals the HPC's enthalpy rise from T24 to T30 divided by mechanical efficiency. All three temperatures are measured. The estimate correlates 0.995 with the simulator's T40.

34 **Why does your spool-balance T40 read about 8% low, and why doesn't that break the monitor?**

The simple balance leaves out turbine cooling air (which dilutes T48), power taken off the spool and the higher heat capacity of hot gas. The data implies a work ratio of 1.315 instead of about 1.01, and it is very stable (coefficient of variation 0.36%). A constant offset cancels because each monitor compares the same formula on measured and healthy values: on healthy flights the monitors stay flat to about 0.01%.

35 **Why do you correct the mass flow with the HPT flow-capacity estimate?**

The guide-vane flow capacity is exactly what the HPT flow modifier describes, and it changes as the HPT degrades. Without the correction, a failing HPT leaked up to 3% into the combustor and nozzle monitors; with the Kalman filter's estimate the worst drift fell to 0.77% and 0.70%.

36 **How do you compute the nozzle's effective area, and is the nozzle choked?**

From compressible-flow continuity: mass flow x sqrt(T50) / (P50 x flow function of the nozzle pressure ratio), with ambient pressure backed out of P2 and flight Mach (it agrees with the standard atmosphere to about 1.2%). The core nozzle pressure ratio is 1.15 to 1.32, below the critical 1.85, so it is never choked in this data and the subsonic flow function applies.

37 **Your failure limits (4% efficiency loss, plus or minus 6% area) aren't learned. Where do they come from?** HARD

They are engineering assumptions, written down in the code: a healthy combustor is about 99% efficient and a 4% loss is treated as failure-level; the design burner loss is about 4.5%, so a 3% drop in burner pressure ratio roughly doubles it; a 6% change in nozzle area means erosion, flap or liner damage. They cannot be learned because N-CMAPSS contains no degraded combustor or nozzle. With real data they would be calibrated against shop findings.

38 **Why did you widen the combustion-efficiency tolerance to 1%?**

Engine fuel-flow meters are accurate to roughly plus or minus 0.5 to 1% of reading, so a smaller apparent efficiency change cannot be told apart from meter error. The physics check prompted the review, because a failing HPT moved the monitor by 0.77%, and we disclose that we changed it.

39 **What do the 19 physics checks prove, and what don't they prove?**

They prove that our formulas conserve energy, obey the second law, give physical efficiencies (0.85 to 0.97) and stay below the Carnot limit; that the monitors are flat on healthy flights and do not react to compressor or turbine wear; and that injected faults are read back at exactly the injected size. They do not prove that real combustor or nozzle faults would be detected on a real engine, or that the limits are right; the injections change one input at a time.

40 **Why do you exclude the HPT from the second-law check?**

The HPT is film-cooled: compressor air joins the gas between stations 40 and 48. Treating it as a single stream makes entropy appear to fall, and our first version flagged 47% of flights for exactly that reason. That was a mistake in the check, not a physics violation. The second law is checked on the uncooled components (fan plus LPC, HPC, burner, LPT, all 100% of flights), and the HPT is covered by the spool energy balance.

Machine learning Questions 41–50

41 **Why tree models instead of deep learning?**

Our inputs are 307 engineered tabular features and we have about 3,700 training flights from 49 engines, which is small for deep networks. Trees train in minutes, are robust, and give exact explanations with TreeSHAP. Temporal information comes from trailing-window features; removing them raises the error from 6.97 to 7.23.

42 **Why stack them with Ridge regression?**

Different model families can make different errors, and a linear blend on out-of-fold predictions is the simplest way to combine them. It also gives a per-model contribution we can show. On validation engines the blend scored 7.05 against 7.26 for the best single model.

43 **Random Forest alone beats your stack on test (6.80 vs 6.97). Why keep the stack?** HARD

Because we chose it on validation engines, where it won, and choosing a model by its test score would be leaking the test set. The honest conclusion is that the ensemble gain is negligible: the difference on test is within noise. For production, a single Random Forest would be simpler and just as good, and we say that openly.

44 **The four models are 0.99 correlated. Is the ensemble adding anything?**

Barely. On raw features the ensemble improved 10.43 to 10.21; the large gains came from the physics features, not from ensembling.

45 **Why does LightGBM get a negative weight in the blend?**

Collinearity. With four nearly identical inputs, Ridge assigns offsetting weights (XGBoost +13.8, LightGBM -8.1), so individual coefficients are not meaningful. The small regularisation chosen on validation (alpha 0.01) allows it; a larger alpha would give steadier weights at a slight validation cost.

46 **What features do you use, and how do you keep them causal?**

307 features: sensor residuals (84), physics ratios and efficiency proxies (72), raw sensor summaries (70), Kalman filter states (56), innovations (14) and flight condition and age (11). Window features (mean, spread, minimum, maximum, slope) cover the trailing five flights. Everything uses only the current and earlier flights of the same engine; imputation values come from training engines.

47 **How did you tune hyperparameters without touching the test set?**

Small grids for each model, scored on validation engines only; XGBoost early-stopped on validation engines; the blend's regularisation chosen on validation engines. The test engines were used once, for the final report.

48 **What's your validation strategy, and why split by engine rather than by flight?**

Train, validation and test are separate engines, stratified by subset, plus five engine-grouped folds for out-of-fold predictions. Splitting by flight would leak: flights of one engine are strongly correlated and share the same end of life, so the model would be tested on engines it had already seen.

49 **Did you try an LSTM, GRU or Transformer? Why did you drop the GRU?**

An earlier version had a GRU branch. The final specification uses four tree models on physics features, and we do not have a like-for-like deep-learning comparison in the final run, so we do not claim one is better. We chose trees for data size, explainability and speed. A 1D CNN or Transformer on the 1 Hz data would be a natural extra base model.

50 **How long does training take, and how long does one prediction take?**

The full pipeline took 71 minutes on Colab, including about 7 minutes of download and aggregation, about 29 minutes of model tuning and about 33 minutes of ablation experiments. Running an engine's whole flight history through the pipeline takes about half a second. In the dashboard, the first load of all 39 engines takes about 9 seconds and a full explanation about 8 seconds.

Results and honesty Questions 51–60

51 **What's your headline accuracy, and in what units?**

RMSE 6.97 flights (cycles), MAE 5.17 flights, R-squared 0.911, on 2,938 flights of 39 held-out NASA test engines across 9 subsets. The 90% range contained the true RUL on 93.4% of those flights.

52 **How does an RMSE of 7 flights compare with published N-CMAPSS results?**

A direct comparison is only valid with the same protocol, and published work varies: many papers use a single subset such as DS02, score only the last cycle, or cap RUL at a maximum. Ours covers nine subsets, every flight, engine-held-out, with no capping. So we present it as an honestly evaluated, competitive number, not as a state-of-the-art claim.

53 **Why is the error lowest near end of life (RMSE 4.2)?**

Near the end, degradation is strong and changing fast, so the signals are clear and the range of possible RUL is small. Early in life, healthy engines look alike, so the prediction leans on age and is less certain (RMSE 8.4).

54 **The model over-predicts on DS02 by about 9 flights. Why, and is that dangerous?** HARD

On DS02 the RMSE is 11.0 with a bias of +8.75: the model says more life is left than there was. Our best explanation is that DS02's failure behaviour or flight mix is under-represented in training, but we have not proven the cause. It would be dangerous if someone planned on the point estimate, which is why planning uses the low end of the range and urgency also uses component health. The fix is per-subset calibration of the range, and with real data, per-fleet calibration.

55 **What does 'positive bias in mid-life' mean for a maintenance planner?**

With 20 to 60 flights left, the model says on average 3.2 flights more than reality. Planners should use the planning RUL, which is the low end of a range built from real errors, so the bias is already allowed for.

56 **How was the 90% range built, and is 93% coverage good or bad?**

From empirical errors: residuals on validation engines plus engine-held-out residuals of training engines, taking the 5th and 95th percentiles separately for predicted RUL up to 20, 20 to 60, and above 60. On test it covered 93.4% (target 90%), so it is slightly conservative, which is the safe side: 97% early in life, 92% in mid-life, 93% at end of life.

57 **Is the range a calibrated probability? Why not?**

No. It is built from error quantiles in three bands, with no per-engine guarantee, and coverage varies by subset. We call it a 90% range, not a probability, and say so in the dashboard.

58 **The top SHAP features are flights flown and hours flown. Isn't the model just learning engine age?** HARD

Partly, yes: engine age is the most informative single input (mean absolute SHAP 6.1 and 5.6 flights), because engines within a subset have similar lifetimes. But the physics matters: raw sensors alone give 10.4, sensor residuals bring it to 7.4, and the full physics and filter features to 6.97. As a block, residuals carry more SHAP importance (12.2) than flight condition and age (11.8). In a real fleet with mixed histories age would be less predictive, which we would have to validate.

59 **Engine 8012 shows 14 flights left when the truth is 0. How do you explain that to a user?**

The RUL model over-predicts on that subset (DS08a, bias +4.4) and this engine's HPT failed quickly at the end. The health layer caught it: the HPT went to Emergency, the engine's urgency went to Emergency, and an alert fired. That is exactly why urgency is the worse of component health and RUL. To the user: trust the red component alert; RUL is one input, not the verdict.

60 **Which of your claims would you not make on the basis of your own results?**

That the ensemble beats single models; that fan, HPT and LPT can be isolated individually; that the range is a calibrated probability; that it works on real engines; that combustor or nozzle detection is validated on real faults; or that it is ready to make maintenance decisions on its own.

Explainability and trust Questions 61–70

61 **How does a user find out why the model gave a particular RUL?**

The 'Why this prediction' tab shows what moved the prediction (TreeSHAP by feature group), which sensors matter (sensor occlusion), the four models behind the blend and how far to trust it. The Gas path tab shows every sensor against a healthy engine.

62 **What is TreeSHAP, and how do you combine it across four models?**

TreeSHAP computes exact Shapley values for tree models: how much each feature pushed this prediction up or down. We compute it for each model, weight each by its effective weight in the Ridge blend, add them up, and group features by their source signal, for example the T50 residual together with its trend statistics.

63 **What is the sensor occlusion view, and how is it different from SHAP?**

We replace one sensor with its healthy-engine value and rerun the whole pipeline, including the Kalman filter; the change in the prediction is that sensor's end-to-end effect. SHAP explains the tree models given their features; occlusion explains the full physics plus ML chain in terms of the sensors an engineer knows.

64 **Why does component health have no SHAP explanation?**

Because it is not a machine-learning output: it is the Kalman filter's physics estimate mapped to a 0-1 scale. Its explanation is the sensor residuals and innovations, and the sensor occlusion view still works for it.

65 **How do you show uncertainty to a non-expert user?**

The planning RUL comes with its range in words ('range 8 to 21 flights'), the chart shades the 90% range, each health value has a plus-or-minus, modules the filter cannot separate are labelled as groups, and caution boxes say in plain language what not to rely on.

66 **What does 'low reliability' on a component mean for the user?**

The filter cannot separate that module from its neighbours (validation error above 10 points), so treat the reading as a group trend and confirm the module with borescope, EGT margin or vibration data before acting.

67 **What would make an engineer stop trusting the tool, and how do you prevent it?**

Unexplained jumps, confident wrong calls and hidden uncertainty. We show uncertainty and explanations, raise a 'model disagreement' alert when tree-only and blended RUL differ by more than 20 flights, flag repeated sensor rejections, run a physics self-check, and state our known biases.

68 **How do you avoid automation bias, where engineers just follow the screen?**

The tool is worded as advisory on every screen; suggested steps are inspections, not removals; acknowledging an alert requires a name; findings are recorded and the model's flag precision is shown back, so engineers see how often it was right.

69 **Is the tool advisory or decision-making? Where is that stated?**

Advisory. The decision panel says 'Decision support only. Your approved maintenance programme governs.', and the same statement is in every API response's list of assumptions.

70 **How would a regulator (DGCA, EASA, FAA) view this tool?**

As a ground-based advisory tool it would sit inside the operator's approved maintenance programme and engineering processes; using it to prioritise work is very different from using it to replace mandated inspections. Any credit such as extended inspection intervals would require validation evidence and regulatory approval, which a simulated-data prototype does not have.

Product and dashboard Questions 71–80

71 **Walk us through the workflow from opening the dashboard to recording an inspection.**

Open the dashboard and the fleet queue is ranked by urgency. Pick the top engine: the overview shows the 3D engine with each module's health, the planning RUL, next steps and alerts. The Gas path tab confirms the physics, the Why tab shows the drivers, and on the Maintenance tab you acknowledge the alert and record the finding, which can be exported as CSV.

72 **How is the fleet queue ranked, and why use urgency rather than RUL alone?**

Urgency is the worse of the component-health band and a planning-RUL level (30 or fewer flights Warning, 15 Alarm, 5 Emergency); ties are broken by planning RUL. Ranking by health band alone hid engine 7009: Normal health but only 6.8 flights of planning RUL, ranked below Warning engines with 10 to 14 flights left.

73 **What does the replay slider do, and why does the demo need it?**

The demo engines are run-to-failure histories, so at their last flight every engine is at end of life. The slider shows the fleet as it was at a chosen share of each engine's flights, which gives a realistic mix of healthy and worn engines: at 50%, 12 Normal, 25 Warning and 2 Alarm.

74 **The 3D model is a turbojet but your data is a turbofan. Isn't that misleading?** HARD

It would be if we hid it, so we don't: the model is labelled 'illustrative turbojet geometry, mapped by axial position'. It is a single-spool turbojet without separate LPC and LPT, so we map its sections to the turbofan modules by position along the axis. All numbers come from the turbofan data; the 3D model is visual only, and the 2D fallback schematic is turbofan-shaped. A real deployment would use the operator's own engine model.

75 **How did you map the 3D model's sections to engine modules?**

The model came from an STL converter as 16 unnamed meshes that each span the whole engine, so names were useless. We measured the radius profile along the axis and rendered a side section, then set boundaries: spinner and first stage (fan), long-blade compressor stages (LPC), short-blade stages (HPC), combustor with injector bosses, two turbine disc groups (HPT, LPT) and the exhaust nozzle. Each vertex is coloured by its axial position.

76 **What triggers a critical alert, and how do you avoid alert fatigue?**

A module entering Alarm (health below 0.55) or Emergency (below 0.30) at the selected flight raises a notification, a pulsing glow on the 3D section and a beep. Each part alerts once per engine and level, Alarm notifications close themselves, an escalation replaces the older notice, and only the three most urgent engines pulse in the queue.

77 **Why colour the engine by temperature by default instead of health?**

Temperature gives immediate physical orientation (cold inlet, hot combustor) and matches the station temperatures shown on the model; it is also the colour scheme of the team's earlier design. Health is on every card, one click away on the model, and critical parts glow in either mode.

78 **What happens when a user uploads their own CSV?**

It accepts raw 1 Hz N-CMAPSS columns or per-flight aggregates, runs the full pipeline, and adds the engine to the fleet queue and the database. Uploading engine 8012's flights reproduces the built-in predictions for that engine to about 1e-14.

79 **How does the system learn from engineers' inspection findings?**

Each finding (confirmed, not found or inconclusive, plus observed health if measured) is stored next to the prediction shown at that flight. The system reports how often its flags were confirmed and how far its health estimates were off, and exports everything as CSV for offline recalibration.

80 **Why doesn't the model update itself automatically from those findings?**

Self-updating models can drift silently, and findings are sparse and biased towards engines that were already flagged. Changes should be versioned, validated and approved before deployment, which our bundle versioning and parity check support.

Engineering and deployment Questions 81–90

81 **What's your tech stack, and why FastAPI plus plain JavaScript?**

The backend is Python with FastAPI, running the same pipeline code as training, so nothing is reimplemented. The frontend is plain JavaScript, SVG and three.js with no build step, which keeps it light and easy to audit. SQLite stores maintenance data with zero setup.

82 **How do you make sure the backend gives exactly the same answers as the training notebook?**

The pipeline module is shared verbatim between notebook and backend, and a test checks they are identical. The bundle exports the notebook's predictions, and a parity check reruns them in the backend: the largest difference is about 1e-13.

83 **How many tests do you have, and what do they cover?**

69 automated backend tests: the pipeline layers, stacking, the API, the maintenance workflow and database, the physics formulas against textbook values, sample uploads and stale uploads after a restart. On top of that, the 19-check physics check and the parity check. Browser behaviour was checked with scripted runs, but those are not yet part of an automated test suite.

84 **You use SQLite with no login. How would this scale to a real fleet with many users?** HARD

It wouldn't, and it isn't meant to: it is a single-presenter demo, and names are typed, not authenticated. For production: PostgreSQL, single sign-on with roles, the API behind a gateway, and a job queue for pipeline runs. The Kalman filter is recursive, so each new flight only needs an incremental update rather than reprocessing the whole history.

85 **How would you connect it to live engine data (ACARS, QAR downloads)?**

Ingest per-flight data from QAR or flight-data downloads, or engine reports sent over ACARS, map the airline's sensor names to ours, and aggregate to the same per-flight format. We need the flight condition (altitude, Mach, throttle, inlet conditions) plus the gas-path sensors. Each new flight then updates the filter state in milliseconds.

86 **What happens if the internet is down? Does the dashboard still work?**

The backend runs locally, but fonts and the three.js library load from the internet. Offline, the 3D engine falls back to the 2D schematic and fonts change; everything else works. Before a demo we would save those libraries into the project.

87 **How do you version models and roll back a bad one?**

Each bundle records its build commit, code hash, library versions and dataset fingerprints, and the model version is shown with predictions. Rolling back means pointing the backend at the previous bundle folder and restarting, with the parity check as the gate.

88 **How would you retrain as new flight data arrives?**

Periodically rerun the training notebook on new run-to-failure or removal data, export a new bundle, run the parity and physics checks, and compare metrics before switching. Inspection findings would recalibrate the ranges and the combustor and nozzle limits.

89 **What are the security risks of the upload feature?**

Very large files (memory), malformed data, no authentication, and no unit or range validation, so a file in the wrong units would be accepted and give nonsense. It is local-only today; production needs size limits, schema and range checks, authentication and sandboxed processing.

90 **How much would it cost to run for a fleet of 100 aircraft?**

We have not costed it, but compute is light: an engine's whole history runs in about half a second on a laptop CPU, so a fleet of about 200 engines fits on one small server. The real costs are data integration, validation and certification work, not computing.

Team, novelty and future Questions 91–100

91 **What's genuinely novel here, beyond combining existing techniques?**

The individual techniques exist. Our contribution is a trustworthy end-to-end system: a physics layer (healthy baseline, Kalman filter, first-principles monitors) feeding ML, an identifiability analysis surfaced to the user, an uncertainty-first decision panel, a physics self-check with fault injection, and a closed maintenance-feedback loop. We were deliberate about stating what it cannot do.

92 **What was the hardest technical problem you solved?**

[Team: choose the one you can tell best.] Strong candidates: discovering that fan, HPT and LPT cannot be separated and designing the product around that; fixing the combustor and nozzle monitors when a failing HPT leaked into them; or processing nine 2-4 GB HDF5 files within Colab's limits.

93 **What would you do differently if you started again?**

Fix the engine-level split and the specification earlier, build the frontend offline-first, use a single Random Forest instead of four models, find a true turbofan 3D model, add automated browser tests from the start, and spend more time on the path to real data.

94 **What's the single biggest weakness of the current system?**

It has only seen simulated engines, so there is no evidence yet on real ones. The second is that fan, HPT and LPT cannot be isolated individually with this sensor set.

95 **What would you need to validate this on real engine data?**

A fleet's per-flight sensor data with flight conditions, its removal history and shop findings, sensor metadata and a few dozen engines; then a back-test against past removals followed by a shadow-mode period alongside engineers.

96 **How would you add fault types (bird strike, foreign-object damage) rather than just gradual wear?**

Abrupt events show up as step changes across several residuals and innovations, so change-point detection on those signals would flag them, and a classifier on the residual signature could name the fault. N-CMAPSS has no such events, so it would need labelled real data.

97 **Could this approach work for other machines (wind turbines, ships, trains)?**

Yes, the pattern generalises: a physics baseline for operating conditions, a state estimator for health, and ML on residuals. It fits industrial gas turbines directly, and wind turbines, marine diesels and rail traction with their own physics models.

98 **What did each team member contribute?**

[Team: fill in one sentence per member, matched to the parts of the system: data and training, physics and Kalman filter, backend, dashboard, evaluation and documentation.]

99 **How did you use AI tools in building this, and how did you check their output?**

[Team: adjust to your actual use.] We used AI coding assistants, including Claude Code, for code, the dashboard and analysis. We checked their output with 69 automated tests, the notebook-to-backend parity check, the 19-check physics check, ablations on held-out engines and our own review of every claim; the judgements about what to claim and what to disclose were ours.

100 **If you had six more months and a real airline partner, what would you build first?**

Validation on the partner's data: back-test against their removals, refit the baseline and ranges, then run in shadow mode with their engineers recording findings. After that, integration with their data feed and maintenance system, then fault classification.

Sources: final bundle trained 31 Oct 2026 (commit 324fe10); test metrics on 2,938 flights of 39 NASA test engines; physics check (python -m backend.physics_check); backend tests (python -m pytest backend/tests). The model and dashboard are a research prototype on simulated data and are not for maintenance decisions.